Pontifícia Universidade Católica de Minas Gerais
Instituto de Ciências Exatas e Informática
Algoritmos e Estruturas de Dados II

Trabalho Prático III

Regras Básicas

1. extends Trabalho Prático 02
2. Neste trabalho, você deverá utilizar o Dataset fornecido no Trabalho Prático 02 como base para os exercícios.

Ordenação Parcial

Para os exercícios de ordenação, seu programa deve criar um arquivo de log na pasta corrente com o nome `matrícula_<algoritmo>.txt` com uma única linha contendo sua matrícula, número de comparações (entre elementos do *array*), número de movimentações (entre elementos do *array*) e o tempo de execução do algoritmo de ordenação. Todas as informações do arquivo de log devem ser separadas por uma tabulação '\t'.

Para os exercícios de árvore binária, seu programa deve criar um arquivo de log na pasta corrente com o nome `matrícula_<algoritmo>.txt` com uma única linha contendo sua matrícula, número de comparações de todas as pesquisas e o tempo de execução total das pesquisas. Todas as informações do arquivo de log devem ser separadas por uma tabulação '\t'.

1. **Ordenação PARCIAL por Seleção em Java**: Refaça a Questão "Ordenação por Seleção" do TP02 considerando a ordenação parcial com k igual a 10. Nome do arquivo de log: `matrícula_selecao_parcial.txt`.
2. **Ordenação PARCIAL por Inserção em C**: Refaça a Questão "Ordenação por Inserção" do TP02 considerando a ordenação parcial com k igual a 10. Nome do arquivo de log: `matrícula_insercao_parcial.txt`.

3. **Ordenação PARCIAL por Quicksort em Java**: Refaça a Questão "Ordenação por Quicksort" do TP02 considerando a ordenação parcial com k igual a 10. Nome do arquivo de log: `matrícula_quicksort_parcial.txt`.

4. **Ordenação PARCIAL por Heapsort em C**: Refaça a Questão "Heapsort" do TP02 considerando a ordenação parcial com k igual a 10. Nome do arquivo de log: `matrícula_heapsort_parcial.txt`.

Estruturas Flexíveis

5. **Lista com Alocação Flexível em C**: Refaça a Questão "Lista com Alocação Sequencial" do TP02 usando lista simplesmente encadeada.

6. **Pilha com Alocação Flexível em Java**: Refaça a Questão "Pilha com Alocação Sequencial" do TP02 usando pilha simplesmente encadeada

7. **Fila com Alocação Flexível em C**: Crie uma Fila de registros. A entrada padrão será como a da questão "Fila Circular com Alocação Sequencial" do TP02, contudo, teremos apenas os comandos `I` para inserir (enfileirar) e `R` para remover (desenfileirar). A saída padrão tem uma linha para cada registro removido, sendo que essa informação será constituída pela palavra "(R)" e o atributo **nome** do registro removido. No final, a saída mostra os registros presentes na Fila do primeitro ao último.

8. **Lista Dupla com Alocação Flexível em Java**: Refaça a Questão "Lista com Alocação Sequencial" do TP02 usando lista duplamente encadeada.

9. **Matriz Dinâmica em Java**: Implemente uma classe denominada Matriz que representa uma matriz flexível que armazena inteiros, conforme visto em sala de aula. No construtor da classe Matriz, dados os números de linha e coluna, faça as devidas alocações de células e as conexões entre elas. Implemente os seguintes métodos:

   - public Matriz somar(Matriz m): efetua a soma da matriz com a matriz m e retorna a matriz resultante.
   - public Matriz multiplicar(Matriz m): efetua a mutiplicação da matriz pela matriz m e retorna a matriz resultante.
   - public void mostrarDiagonalPrincipal(): Imprime os elementos da diagonal principal da matriz separados por espaço.

- public void mostrarDiagonalSecundaria(): Imprime os elementos da diagonal secundária da matriz separados por espaço.

A entrada padrão é composta por vários casos de teste, sendo que o número de casos é um inteiro contido na primeira linha da entrada. Em seguida, temos cada um dos casos de teste, sendo que cada caso é composto por duas matrizes. Para cada caso de teste, temos a primeira linha contendo dois inteiros, representando os números de linhas e de colunas da primeira matriz, respectivamente. Em seguida, temos os elementos da primeira matriz que estão representados nas próximas $l1$ linhas, onde $l1$ é o número de linhas da primeira matriz, tendo cada linha $c1$ colunas, onde $c1$ é o número de colunas da primeira matriz. Na linha seguinte, temos os elementos da segunda matriz que estão representados nas próximas $l2$ linhas, onde $l2$ é o número de linhas da segunda matriz, tendo cada linha $c2$ colunas, onde $c2$ é o número de colunas da segunda matriz.

A saída padrão apresenta a seguinte configuração para cada caso de teste: as duas primeiras linhas de saída correspondem, respectivamente, à diagonal principal da primeira matriz e à diagonal secundária da segunda matriz. Em seguida, apresenta-se a matriz resultante da soma das duas matrizes, com ls linhas e cs colunas. Por fim, apresenta-se a matriz resultante da multiplicação entre a primeira e a segunda matriz, com lm linhas e cm colunas.

10. **Ordenação por Seleção com Lista Flexível em C**: Refaça a Questão "Ordenação por Seleção" do TP02 usando lista simplesmente encadeada. Nome do arquivo de log: `matrícula_selecao_flexivel.txt`.

11. **Ordenação por Quicksort com Lista Dupla Flexível em Java**: Refaça a Questão "Ordenação por Quicksort" do TP02 usando lista duplamente encadeada. Nome do arquivo de log: `matrícula_quicksort_flexivel.txt`.

Árvores Binárias

12. **Árvore Binária em Java**: Crie uma Árvore Binária, fazendo inserção de registros conforme a entrada padrão. A chave de pesquisa é o atributo **nome**. Não insira um elemento se sua chave estiver na árvore. Em seguida, pesquise se alguns registros estão na árvore, mostrando seus respectivos caminhos de pesquisa. A entrada padrão possui o mesmo formato da questão "Pesquisa Sequencial" do TP02. Os registros referentes à primeira parte da entrada devem ser inseridos na árvore. A segunda parte da entrada contém as chaves a serem pesquisadas.

    A saída padrão é composta por várias linhas, uma para cada pesquisa. Cada linha é composta pelo caminho de ponteiros (`raiz, esq ou dir`) utilizados na pesquisa e, no final, pelas

palavras SIM ou NAO, indicando o resultado da pesquisa. Ao final, os registros presentes na árvore devem ser apresentados usando caminhamento em ordem. Nome do arquivo de log: `matrícula_arvore_binaria.txt`.

13. **Árvore Binária em C**: Repita o exercício anterior "Árvore Binária", realizando a implementação em C. Nome do arquivo de log: `matrícula_arvore_binaria.txt`.